

# Layer Struct 项目架构设计文档

对话记录 · 采集 workflow · Memory RAG · 用户画像 · 上下文压缩 · Skill Orchestrator

生成日期：2026-05-15

**一句话定位：把当前系统从“能查资料的聊天页”，升级为“可长期积累、可追溯、可继续对话的个人采集中心”。**

## 1. 目标与设计判断

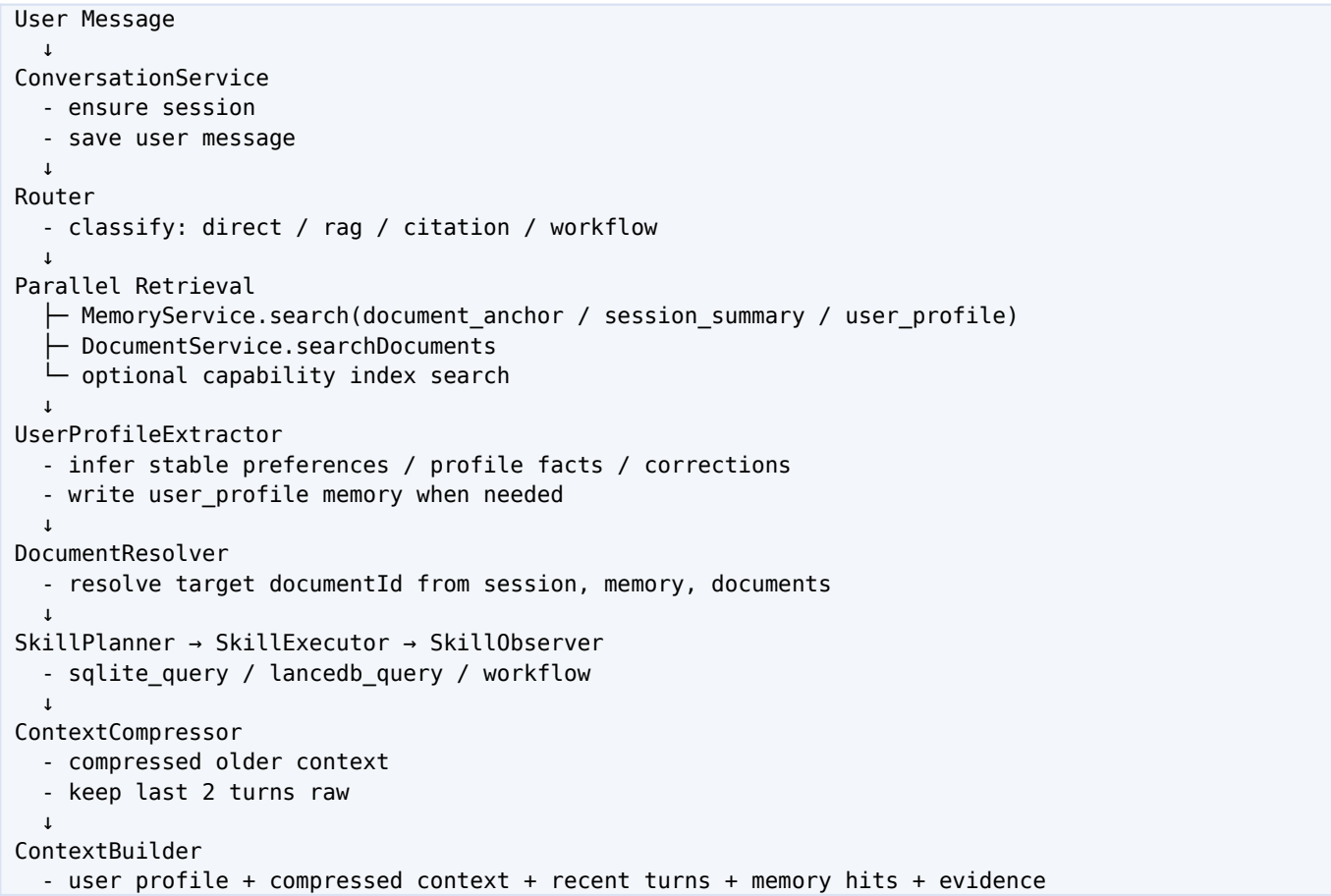
本文件基于当前 layer-struct 代码结构和前序讨论整理，目标不是再堆一个 RAG 模块，而是把系统对象模型、运行链路和长期记忆能力定清楚。核心判断如下：

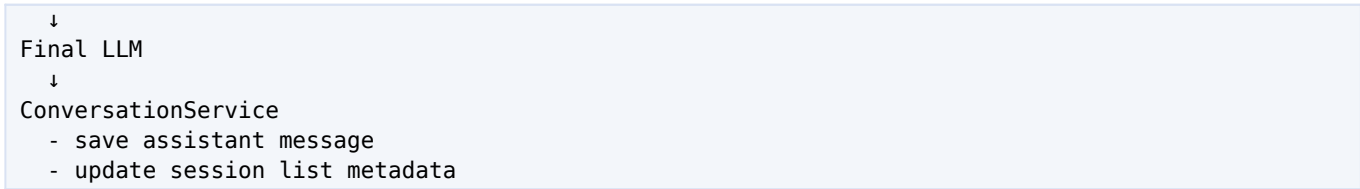
- 对话记录和采集信息必须分层：conversation\_messages 不是 documents/chunks，聊天历史不能和被采集资料混在一起。
- Memory RAG 负责找线索和锚点，例如“用户之前保存过哪篇文章”“某个实体对应哪个 documentId”；Document RAG 才负责提供原文证据。
- 用户画像是长期 Memory 的特殊类型，作用于最终大模型的风格、约束、偏好和上下文，不应被当作事实证据或原文引用。
- 上下文压缩不是普通摘要，而是结构化保真压缩：保留目标、事实、决策、未解决问题、documentId、用户纠错和工作流结果。
- Router、SkillPlanner、MemoryResolver、DocumentResolver、ContextCompressor 必须各司其职，避免让大模型一边猜路径一边回答。

## 2. 当前代码状态简述

| 模块              | 当前状态                                                                         | 问题/下一步                                                        |
|-----------------|------------------------------------------------------------------------------|---------------------------------------------------------------|
| Router          | 已接入 AI_ROUTER_MODEL，失败会走规则兜底；目前需要补 routerTrace 可观测性。                         | 明确记录 router 来源：model / rules / model_failed / not_configured。 |
| DocumentService | 已支持 documents/chunks 写入、LanceDB 写入、EvidencePack expansion。                   | 需要与 collected_items 工作流绑定，形成采集对象层。                            |
| MemoryService   | 已有 document_anchor memory、memory_items SQLite、LanceDB memory table、命中加分和衰减。  | 需要新增 user_profile 长期画像 memory，并接入最终 prompt。                   |
| Conversation    | 已有 conversation_messages，但缺少 conversation_sessions 和 conversation_summaries。 | 需要对话列表、继续对话、压缩上下文。                                            |
| Frontend        | 当前单聊天页 + details；sessionId 仍是一次性 randomUUID。                                 | 需要左侧对话列表、会话切换、历史消息恢复。                                         |
| ContextBuilder  | 已压缩 RoutePlan/SkillPlan/EvidencePack 输出；已支持 expandedItems。                   | 需要加入 compressedContext + 最近两轮原文 + userProfile。                |

## 3. 总体架构

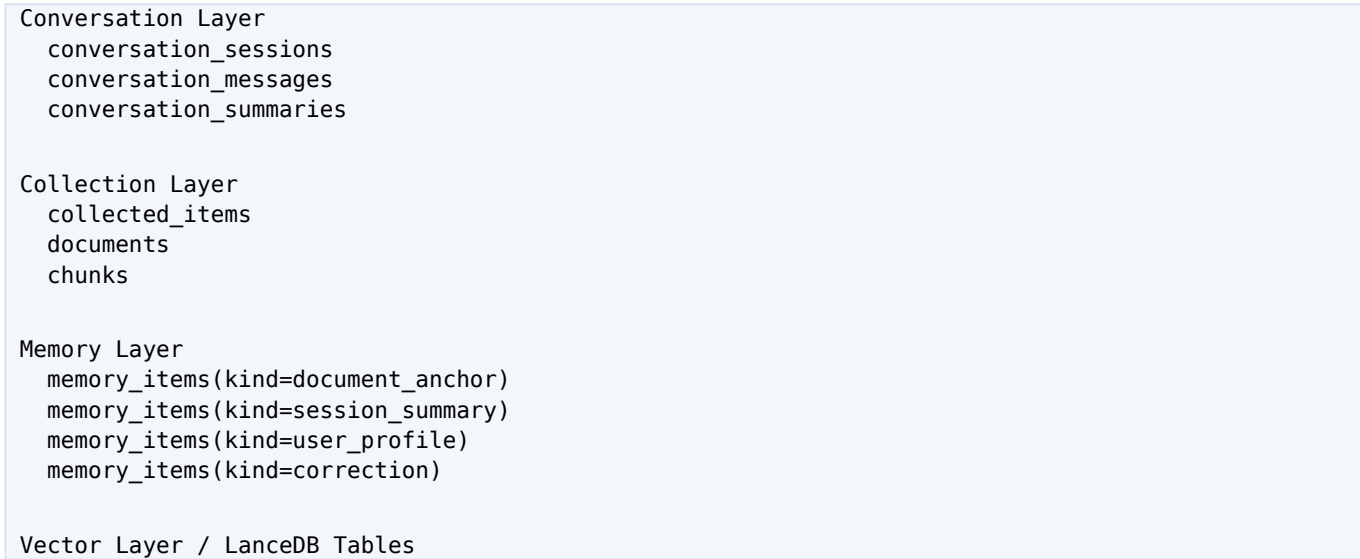




## 4. 核心对象模型

| 对象                  | 用途                                                      | 关键字段                                                                                  |
|---------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------|
| ConversationSession | 对话会话，用于列表、继续对话、标题和最近更新时间。                               | id, userId, projectId, title, status, messageCount, lastMessagePreview, lastMessageAt |
| ConversationMessage | 对话消息，只记录用户/助手/系统消息，不作为采集资料。                             | id, sessionId, runId, role, content, contentType, metadata, tokenEstimate, createdAt  |
| ConversationSummary | 较早对话的结构化保真压缩结果。                                         | sessionId, fromMessageId, toMessageId, summaryJson, summaryText, model                |
| CollectedItem       | 产品层采集对象，代表一篇文章、一段文本、URL、转写等。                            | id, kind, title, source, status, documentId, contentHash, metadata                    |
| Document / Chunk    | RAG 原文证据层，用于引用和具体段落。                                    | documentId, title, source, chunkId, chunkIndex, content                               |
| MemoryItem          | 长期记忆，包括 document_anchor、session_summary、user_profile 等。 | kind, documentId, content, summary, entities, topics, score, hitCount, isPinned       |
| UserProfileMemory   | 长期用户画像，作用于最终回答风格和约束。                                    | profileKey, value, confidence, sourceMessageId, lastVerifiedAt, stability             |

## 5. 数据分层原则



```
document_chunks
memory_items
session_summaries
capability_index
```

Rule

```
conversation history != collected content
memory != original evidence
user_profile != factual document source
```

## 6. 对话记录与继续对话

目前只有 conversation\_messages，无法支撑完整的历史会话管理。需要新增 conversation\_sessions，所有聊天请求都先 ensure session，再写入消息。前端通过 API 拉取会话列表，点击某个会话后带同一个 sessionId 继续对话。

| API                                        | 说明                                              |
|--------------------------------------------|-------------------------------------------------|
| GET /api/conversations                     | 查询对话列表，按 updated_at 倒序。                         |
| POST /api/conversations                    | 创建新对话，可传 title/userId/projectId。                |
| GET /api/conversations/:sessionId          | 读取会话元数据。                                        |
| GET /api/conversations/:sessionId/messages | 读取该会话消息。                                        |
| POST /api/conversations/:sessionId/archive | 归档对话。                                           |
| POST /api/chat/stream                      | 如果传 sessionId 则继续该对话；否则自动创建新会话并返回 conversation。 |

## 7. 采集信息入库 workflow

新增 workflow.ingest\_collected\_content，作为通用采集入库 workflow。公众号文章 workflow 只负责抓取，拿到正文后复用这条链路。

```
workflow.ingest_collected_content
1. create collected_item(status=pending)
2. update collected_item(status=processing)
3. write documents/chunks
4. write LanceDB document_chunks
5. create document_anchor memory
6. update session_state.currentDocumentId
7. update collected_item(status=completed, documentId)
8. return collectedItemId / documentId / chunkCount / memoryId
```

| 场景               | kind           | 备注                        |
|------------------|----------------|---------------------------|
| 用户粘贴一段资料并说“保存这段” | manual_text    | 标题可由用户给出或自动取首行。           |
| 公众号文章链接          | wechat_article | WeSpy 抓取后复用通用入库链路。        |
| URL 页面           | url            | 后续可接网页正文提取器。              |
| 音视频转写文本          | transcript     | 后续可接 Whisper / 本地 ASR。    |
| 临时笔记             | note           | 短文本也可以形成 document_anchor。 |

## 8. Memory RAG 与用户画像长期记忆

新增用户画像长期 Memory。它不是 document\_anchor，也不是 session\_summary，而是基于用户长期表达、纠错、偏好和稳定身份信息形成的 profile memory。它应该由推理小模型提取，但必须经过严格门控，避免把一次性的情绪、临时任务或误判写成长期画像。

| Memory kind     | 作用                       | 是否可作为证据                      |
|-----------------|--------------------------|------------------------------|
| document_anchor | 定位用户保存过的文档和 documentId。  | 否，只能导航。                      |
| session_summary | 跨会话延续讨论目标和上下文。           | 否，只能理解对话历史。                  |
| user_profile    | 记录用户长期偏好、身份、工作方式、输出风格约束。 | 否，只作用于回答策略。                  |
| correction      | 记录用户纠错，避免重复犯错。           | 否，但可作为模型行为约束。                |
| workflow_trace  | 记录重要工作流结果，例如采集成功、失败原因。   | 否，结果需回查 collected/documents。 |

### 8.1 用户画像提取规则

- 由小模型/推理模型在每轮用户消息后异步或同步轻量判断：是否存在稳定画像信息。
- 只提取长期稳定信息，例如角色、偏好、反复出现的约束、明确纠错、技术栈偏好、输出格式偏好。
- 不提取短期任务状态、单次情绪、未经确认的敏感推断。
- 每条 profile memory 必须有 confidence、sourceMessageId、createdAt、lastUsedAt。
- 用户画像进入最终 prompt 时必须标注：这是用户偏好/画像，不是事实证据。

```
UserProfileMemory example
{
  kind: "user_profile",
  profileKey: "response_style",
  value: "用户偏好直接、少废话、给具体方案和 Codex 指令。",
  confidence: 0.92,
  source: "explicit_user_preference",
  stability: "long_term",
  isPinned: true
}
```

## 9. 上下文压缩策略

最终大模型不应该读取全部历史原文，也不应该只读最近 8 条。建议使用“结构化保真压缩 + 最近两轮原文”的组合。

```
Final LLM Context
System instruction
↓
User profile memory compact
↓
```

Conversation compressed context  
↓  
Recent raw turns: last 2 user+assistant turns  
↓  
MemoryHits compact  
↓  
RoutePlan / SkillPlan / Observation compact  
↓  
EvidencePack expanded chunks  
↓  
Current user request

## 9.1 保真压缩 JSON Schema

```
{
  "userGoals": [],
  "facts": [],
  "decisions": [],
  "openQuestions": [],
  "referencedDocuments": [
    { "documentId": "...", "title": "...", "source": "...", "reason": "..." }
  ],
  "userPreferences": [],
  "corrections": [],
  "workflowResults": [],
  "importantMessages": [
    { "role": "user", "content": "...", "reason": "..." }
  ]
}
```

## 10. 模型职责划分

| 模型/能力            | 配置                                    | 职责                                     | 输出                                 |
|------------------|---------------------------------------|----------------------------------------|------------------------------------|
| Router Model     | AI_ROUTER_MODEL                       | 判断任务类型、是否 RAG、是否 Workflow、是否 citation。 | RoutePlan JSON                     |
| Compressor Model | AI_COMPRESSOR_MODEL 或 AI_ROUTER_MODEL | 压缩较早对话为结构化上下文。                         | ConversationCompressedContext JSON |
| Profiler Model   | 可复用 AI_COMPRESSOR_MODEL               | 从用户消息中提取长期画像候选。                        | UserProfileCandidate JSON          |
| Embedding Model  | AI_EMBEDDING_MODEL                    | 文档 chunk、memory、session summary 向量化。   | vector                             |
| Final Chat Model | AI_CHAT_MODEL                         | 基于上下文、证据、画像和约束生成最终回答。                  | Markdown answer                    |

## 11. RAG 与检索链路

Parallel Retrieval  
memory\_items vector search  
memory\_items keyword search

```
sqlite document search
lancedb document_chunks search
session_summaries search
```

```
DocumentResolver
  session_state.currentDocumentId
  document_anchor memory
  sqlite documents
  recent documents
  lancedb fallback
```

```
Evidence Expansion
  hit chunk k
  read chunk k-1, k, k+1
  send expandedContent to final model
```

关键边界：Memory 只能定位 documentId，原文引用必须来自 chunks/document\_chunks。用户画像只能影响回答策略，不能成为事实来源。

## 12. UI 结构建议

|                                                                                                                            |                                                                                                                                |                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Conversation List <ul style="list-style-type: none"><li>- New chat</li><li>- sessions</li><li>- active highlight</li></ul> | Chat + Run Progress <ul style="list-style-type: none"><li>- messages</li><li>- execution timeline</li><li>- composer</li></ul> | Details / Collected <ul style="list-style-type: none"><li>- routePlan</li><li>- skillPlan</li><li>- memoryHits</li><li>- evidencePack</li></ul> |
|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|

- 左侧展示全部对话，点击恢复 sessionId 和消息。
- 中间展示当前对话和执行过程，progress 不写入 conversation\_messages 正文。
- 右侧保留 details，并增加采集条目/调试信息 tab。
- 发送消息时必须带 currentSessionId；如果没有，后端自动创建并返回 conversation。

## 13. API 设计

| 分类           | API                                        | 说明              |
|--------------|--------------------------------------------|-----------------|
| Conversation | GET /api/conversations                     | 列表展示全部会话。       |
| Conversation | POST /api/conversations                    | 新建会话。           |
| Conversation | GET /api/conversations/:sessionId/messages | 恢复某个会话的全部或分页消息。 |
| Conversation | POST /api/conversations/:sessionId/archive | 归档会话。           |
| Collected    | POST /api/collected                        | 直接创建采集条目并入库。    |
| Collected    | GET /api/collected                         | 列出采集信息。         |
| Chat         | POST /api/chat/stream                      | 继续对话或自动创建会话；返回  |

|        |                 |                                                         |
|--------|-----------------|---------------------------------------------------------|
|        |                 | conversation、memoryHits、context。                        |
| Health | GET /api/health | 展示 router/chat/embedding/compressor/profile model 配置状态。 |

## 14. SQLite 表设计摘要

| 表                             | 用途                                                    |
|-------------------------------|-------------------------------------------------------|
| conversation_sessions         | 会话列表与继续对话入口。                                          |
| conversation_messages         | 对话消息，保留 role、contentType、metadata、runId。              |
| conversation_summaries        | 对话历史的结构化压缩结果。                                         |
| collected_items               | 采集信息产品层对象。                                            |
| documents                     | 采集内容正文。                                               |
| chunks                        | 采集内容切片，原文证据源。                                         |
| memory_items                  | 长期记忆，包括 document_anchor、session_summary、user_profile。 |
| runs / run_steps / run_events | 执行过程和可观测性。                                            |
| route_logs                    | Router 判断日志。                                          |

## 15. 实施路线

| 阶段 | 目标                  | 交付                                                   |
|----|---------------------|------------------------------------------------------|
| P0 | Router 可观测性         | routerTrace、health 展示模型、jsonMode 开关。                 |
| P1 | Conversation 系统     | sessions/messages/summaries API + 左侧会话列表。            |
| P2 | Collected Workflow  | workflow.ingest_collected_content + collected_items。 |
| P3 | Context Compression | ContextCompressor + 最近两轮原文 + summary 持久化。            |
| P4 | User Profile Memory | 画像提取、画像记忆、画像注入最终 prompt。                             |
| P5 | 质量与治理               | 记忆衰减、画像置信度、删除/归档、调试面板。                               |

## 16. Codex 指令

下面是一段可直接交给 Codex 的实现指令。建议一次不要全部做完，可按 P1-P4 分阶段执行。

请继续修改当前 TypeScript 项目，实现“完整对话系统 + 采集工作流 + 上下文压缩 + 用户画像长期 Memory”。保持当前 Node.js http + TypeScript + SQLite + LanceDB 架构，不引入 Express/React。npm run build 必须通过。

### 一、Conversation 系统



1. 在 types.ts 增加 ConversationSession、ConversationMessage、ConversationSummary、ConversationCompressedContext、ConversationContextPack。
2. 在 sqliteStore.ts 新增 conversation\_sessions、conversation\_summaries 表，并为 conversation\_messages 增加 run\_id、content\_type、metadata\_json、token\_estimate 字段，兼容旧表 ALTER。
3. 新增 ConversationService: ensureSession、createSession、listSessions、getSessionWithMessages、appendMessage、archiveSession、getRecentMessages、getMessagesForCompression。
4. Orchestrator 不再直接 sqlite.insertMessage，全部通过 ConversationService 写入 user/assistant message。
5. /api/chat 和 /api/chat/stream: 如果传 sessionId 则继续该对话；如果不传则自动创建会话，并在响应/done 事件中返回 conversation。

## 二、前端会话列表

1. public/index.html 改成左侧会话列表 + 中间聊天 + 右侧 details。
2. public/app.js 去掉 const sessionId = crypto.randomUUID(), 改成 let currentSessionId。
3. 页面加载 GET /api/conversations; 点击会话 GET /api/conversations/:id/messages 渲染历史消息。
4. 新建对话按钮 POST /api/conversations, 清空消息区并切换 currentSessionId。
5. 发送消息时带 currentSessionId; done 返回 conversation 后刷新左侧列表。

## 三、采集信息 workflow

1. types.ts 增加 CollectedItem、CollectedItemKind。
2. sqliteStore.ts 新增 collected\_items 表和 create/update/get/list 方法。
3. 新增 CollectedContentWorkflow:  
ingest({kind,title,source,content,metadata,userId,sessionId,projectId,onProgress})。
4. ingest 流程: create collected\_item pending -> processing -> writeDocument -> create document\_anchor memory -> upsertSessionState -> completed。
5. 新增 capability workflow.ingest\_collected\_content; Router 将“保存这段/采集/入库/保存到知识库/存为资料”路由到该 workflow。
6. workflow.ingest\_wechat\_article 抓取文章后复用 CollectedContentWorkflow, kind=wechat\_article。

## 四、上下文压缩

1. env.ts 增加 AI\_COMPRESSOR\_MODEL、CONTEXT\_RAW\_RECENT\_TURNS=2、CONTEXT\_COMPRESS\_MIN\_MESSAGES=6、CONTEXT\_COMPRESS\_MAX\_MESSAGES=30。
2. 新增 ContextCompressor: 读取较早消息，调用小模型压缩为 ConversationCompressedContext JSON，存 conversation\_summaries。
3. 最近两轮 user+assistant 原文必须完整保留，不参与压缩。
4. 如果 compressor 不可用，不阻断主流程，直接返回最近消息。
5. contextBuilder.ts 的 FinalContext 增加 conversationContext，并在 prompt 中按顺序注入：用户画像、压缩上下文、最近两轮原文、memoryHits、routePlan、skillPlan、evidencePack、当前请求。

## 五、用户画像长期 Memory

1. types.ts 增加 UserProfileCandidate / UserProfileMemory 或在 MemoryItem.metadata 中支持 profileKey、value、confidence、stability、sourceMessageId。
2. 新增 UserProfileExtractor，使用 AI\_COMPRESSOR\_MODEL 或 AI\_ROUTER\_MODEL，从用户新消息中提取长期画像候选。
3. 只允许提取稳定、明确、可复用的信息：身份角色、长期偏好、输出风格、技术偏好、明确纠错。禁止提取单次情绪、敏感推断、临时任务。
4. 提取结果写入 memory\_items(kind=user\_profile)，重复 profileKey 做合并更新，不盲目新增。
5. 每次生成前检索 user\_profile memory，形成 UserProfile Compact，注入最终 prompt。明确：用户画像不是事实证

据，只影响回答策略和约束。  
6. 命中 user\_profile 后 markMemoryHit，参与记忆曲线。

六、API

新增：  
GET /api/conversations  
POST /api/conversations  
GET /api/conversations/:sessionId  
GET /api/conversations/:sessionId/messages  
POST /api/conversations/:sessionId/archive  
GET /api/collected  
POST /api/collected  
GET /api/collected/:id

七、验收

- 1. 打开页面能看到历史会话列表。
- 2. 点击旧会话能恢复消息并继续对话，sessionId 不变。
- 3. 保存一段内容会创建 collected\_item、document、chunks、LanceDB vectors、document\_anchor memory。
- 4. 超过 6 条消息后生成 conversation\_summary，最近两轮仍保留原文。
- 5. 用户说“以后回答少废话，直接给方案和 Codex 指令”，系统写入 user\_profile memory；后续新会话也能影响回答风格。
- 6. 用户要原文引用时，不能引用 user\_profile 或 memory summary，必须引用 document chunks。
- 7. npm run build 通过。

# 17. 风险与边界

| 风险           | 处理方式                                                       |
|--------------|------------------------------------------------------------|
| 用户画像误写       | 必须有 confidence、sourceMessageld、profileKey；只写明确长期偏好和纠错。     |
| 压缩丢信息        | 最近两轮原文永远保留；摘要采用结构化 JSON；重要 documentId 不可丢。                 |
| Memory 被当证据  | Prompt 和代码层都要约束：memory 只导航，原文证据只来自 chunks。                 |
| 会话列表污染采集库    | Conversation 和 Collected 分表，消息不直接进入 document RAG，除非用户明确采集。 |
| Prompt 过大    | 压缩历史、裁剪 expanded evidence、只注入 compact profile。             |
| Router 模型不可见 | routerTrace + health + route_logs 显示 model/rules/failure。  |

# 18. 最终形态

系统最终不是一个“聊天机器人”，而是一个会持续积累资料、记忆偏好、理解历史上下文，并能把采集信息转化为可引用证据的个人知识工作台。

Short-term continuity: ConversationSession + recent raw turns  
Long-term context: ConversationSummary + MemoryItems  
User adaptation: UserProfileMemory

Original evidence: Documents + Chunks + ExpandedEvidence  
Workflow traceability: Runs + Steps + Events  
Product object: CollectedItem